Space Empire

Supplément — Système multi-univers

Complément au document de conception technique v1

Mars 2026

1. Concept et règles fondamentales

Le jeu supporte plusieurs univers indépendants, chacun avec ses propres règles de jeu. Un univers est un espace de jeu isolé — les joueurs, planètes, flottes et guildes d'un univers n'interagissent jamais avec ceux d'un autre.

Règle	Détail
Isolation totale	Impossible de se déplacer d'un univers à l'autre
Multi-inscription	Un joueur peut être inscrit dans plusieurs univers simultanément
Ressources communes	Les types de ressources sont identiques dans tous les univers
Carte unique par univers	Galaxies, systèmes et planètes sont générés spécifiquement pour chaque univers
Guildes par univers	Une guilde existe dans un univers spécifique, pas au niveau du compte joueur
Univers temporaire	Un univers peut avoir une date de fin au-delà de laquelle il devient inaccessible
Fermeture	À la fermeture, les données sont conservées mais le joueur ne peut plus y accéder

1.1 Modes de jeu — variantes d'univers

Chaque univers est associé à un fichier YAML de configuration qui définit ses règles spécifiques. Les variantes possibles incluent :

- Multiplicateurs de temps différents par action (recherche x10, construction vaisseaux x2, production ressources x1...)
- Désactivation des guildes
- Activation de la conquête de planètes (voler la planète d'un autre joueur)
- Toute autre variante de gameplay définie dans le YAML

> *Les paramètres de jeu sont dans config/game/universes/*.yaml. La carte (galaxies, systèmes, planètes) est dans config/universes/*.json, générée par un script Python externe.*

2. Modèle de données

2.1 Vue d'ensemble

```
Player                              # Compte global du joueur (un seul compte)
├── pseudo, email, nom, prénom...
└── current_player_universe: PlayerUniverse|null   # univers actif courant

Universe                            # Définition d'un univers
├── slug: string                    # clé vers les fichiers YAML et JSON
├── name: string
├── status: pending | active | closed
├── starts_at: datetime|null
└── ends_at: datetime|null          # null = permanent

PlayerUniverse                      # Inscription d'un joueur dans un univers
├── player: Player
├── universe: Universe
└── joined_at: datetime

# Tout ce qui était lié à Player est maintenant lié à PlayerUniverse :
Fleet               → player_universe: PlayerUniverse
Guild               → universe: Universe
GuildMember         → player_universe: PlayerUniverse
PlayerTechnology    → player_universe: PlayerUniverse
ResearchQueue       → player_universe: PlayerUniverse
AllianceContract    → player_universe_a + player_universe_b
PlayerDiscovery     → player_universe: PlayerUniverse

# La carte est rattachée à l'univers :
Galaxy              → universe: Universe
SolarSystem         → galaxy: Galaxy
Planet              → solar_system: SolarSystem
```

2.2 Gestion du changement d'univers

Le joueur peut changer d'univers actif depuis une page de sélection. Le champ current_player_universe sur Player indique l'univers dans lequel il joue actuellement.

```
# Flux de changement d'univers

current_player_universe = null (ou univers fermé)
→ redirection automatique vers la page de sélection des univers
```

```
Le joueur sélectionne un univers :
├── PlayerUniverse existe déjà (joueur déjà inscrit)
│   └── on le set directement comme current_player_universe
└── PlayerUniverse n'existe pas (première fois dans cet univers)
    └── on crée le PlayerUniverse, puis on le set comme current_player_universe
```

> *La redirection est gérée par un EventListener Symfony qui vérifie current_player_universe à chaque requête. Si null ou univers fermé, il redirige vers la sélection avant que le controller ne soit atteint.*

2.3 Fermeture d'un univers

- Un job RabbitMQ est planifié à ends_at pour passer Universe.status à closed.
- Les données (PlayerUniverse, flottes, planètes...) sont conservées en BDD — aucune suppression.
- Le joueur ne peut plus accéder à l'univers fermé. UniverseContext lève une exception si status = closed, interceptée par l'EventListener qui redirige vers la sélection.
- Si current_player_universe pointe vers un univers fermé, le joueur est renvoyé à la sélection et devra en choisir un autre.

3. Configuration des univers

3.1 Structure des fichiers

```
config/
├── universes/                      # Cartes générées par Python (JSON)
│   ├── omega-1.json
│   └── turbo-1.json
└── game/
    ├── universes/                  # Paramètres de jeu (YAML)
    │   ├── omega-1.yaml
    │   └── turbo-1.yaml
    ├── ships/
    └── ...
```

3.2 Fichier YAML — paramètres de jeu

```yaml
# config/game/universes/turbo-1.yaml
slug: turbo-1
name: "Univers Turbo"

features:
  guilds_enabled: false             # pas de guildes dans cet univers
  planet_conquest: false            # impossible de voler une planète

multipliers:
  research_time: 0.1               # x10 plus rapide (on divise par ce coeff)
  ship_build_time: 0.5             # x2 plus rapide
  resource_production: 1.0         # production normale
  travel_time: 1.0                 # déplacements normaux

limits:
  max_planets: 5
  max_fleet_size: 100

temporary: true
```

> *Un multiplicateur < 1 accélère (ex: 0.1 = x10 plus rapide). Un multiplicateur > 1 ralentit (ex: 2.0 = x2 plus lent). La valeur 1.0 correspond au comportement standard.*

3.3 Fichier JSON — carte de l'univers

```
// config/universes/omega-1.json  (généré par scripts/universe_generator.py)
{
  "slug": "omega-1",
  "name": "Univers Omega",
  "galaxies": [
    {
      "name": "Galaxie Alpha",
      "solar_systems": [
        {
          "coord_x": 3,
          "coord_y": 5,
          "planets": [
            { "position": 1 },
            { "position": 4 },
            { "position": 7 }
          ]
        },
        {
          "coord_x": -2,
          "coord_y": 8,
          "planets": [
            { "position": 3 },
            { "position": 6 }
          ]
        }
      ]
    },
    {
      "name": "Galaxie Beta",
      "solar_systems": []
    }
  ]
}
```

> *Le JSON contient uniquement la carte (positions, planètes). Les paramètres de gameplay restent dans le YAML. Les deux fichiers partagent le même slug pour faire la liaison.*

4. Import de la carte au démarrage

4.1 Principe — commande idempotente

Une commande Symfony est lancée via le script Docker entrypoint à chaque démarrage du serveur. Elle est idempotente : peu importe combien de fois elle est lancée, elle ne modifie jamais ce qui existe déjà en BDD.

Idempotente signifie qu'exécuter la commande plusieurs fois produit toujours le même résultat. Si quelque chose plante à mi-import, on relance sans risque de doublon ou de corruption.

```
Pour chaque fichier dans config/universes/*.json :

├── Universe absente en BDD ?
│   └── créer Universe + toutes ses galaxies + systèmes + planètes
│
└── Universe présente en BDD ?
    └── Pour chaque galaxie dans le JSON :
        ├── Galaxie présente en BDD ? → skip (on ne touche à rien)
        └── Galaxie absente en BDD  ? → créer galaxie + systèmes + planètes

# Résultat : seul le contenu absent est ajouté.
# Ce qui existe reste intact, peu importe le nombre de lancements.
```

4.2 La commande Symfony

```
// src/Command/UniverseImportCommand.php

#[AsCommand(name: 'app:universe:import')]
class UniverseImportCommand extends Command
{
    protected function execute(InputInterface $input, OutputInterface $output):
int
    {
        foreach (glob($this->configDir . '/universes/*.json') as $file) {
            $data     = json_decode(file_get_contents($file), true);
            $slug     = $data['slug'];

            // Récupère l'univers existant ou le crée
            $universe = $this->universeRepository->findOneBy(['slug' => $slug])
                ?? $this->universeManager->createFromData($data);

            // Check galaxie par galaxie
```

```
            foreach ($data['galaxies'] as $galaxyData) {
                $exists = $this->galaxyRepository->findOneBy([
                    'universe' => $universe,
                    'name'     => $galaxyData['name'],
                ]);

                if ($exists) {
                    $output->writeln("  [{$galaxyData['name']}] déjà présente,
skip.");
                    continue;
                }

                // Nouvelle galaxie → import complet (systèmes + planètes)
                $this->universeImporter->importGalaxy($universe, $galaxyData);
                $output->writeln("  [{$galaxyData['name']}] importée.");
            }
        }

        return Command::SUCCESS;
    }
}
```

4.3 Script Docker entrypoint

```
#!/bin/bash
# docker/entrypoint.sh

# 1. Migrations Doctrine (création / mise à jour des tables BDD)
php bin/console doctrine:migrations:migrate --no-interaction

# 2. Import des univers (idempotent — sans danger à chaque démarrage)
php bin/console app:universe:import

# 3. Démarrage du serveur PHP
php-fpm
```

5. UniverseContext — service central

UniverseContext est le service Symfony qui expose l'univers courant et sa configuration à tous les autres services. C'est le point d'entrée unique pour tout ce qui dépend de l'univers actif du joueur.

```
// src/Service/Universe/UniverseContext.php

class UniverseContext
{
    /**
     * Retourne le PlayerUniverse actif.
     * Lève UniverseNotAccessibleException si null ou univers fermé.
     * → intercepté par EventListener → redirection sélection univers
     */
    public function getPlayerUniverse(): PlayerUniverse
    {
        $pu = $this->security->getUser()?->getCurrentPlayerUniverse();

        if (!$pu || $pu->getUniverse()->getStatus() === UniverseStatus::CLOSED) {
            throw new UniverseNotAccessibleException();
        }

        return $pu;
    }

    /**
     * Charge la configuration YAML de l'univers courant.
     */
    public function getConfig(): UniverseConfig
    {
        $slug = $this->getPlayerUniverse()->getUniverse()->getSlug();
        return $this->universeCatalog->get($slug);
    }

    /**
     * Retourne un multiplicateur de l'univers courant.
     * ex: getMultiplier('research_time') → 0.1 pour un univers x10
     */
    public function getMultiplier(string $key): float
    {
        return $this->getConfig()->getMultiplier($key);
    }
```

```
    /**
     * Vérifie si une feature est activée dans l'univers courant.
     * ex: isFeatureEnabled('guilds_enabled')
     */
    public function isFeatureEnabled(string $feature): bool
    {
        return $this->getConfig()->isFeatureEnabled($feature);
    }
}
```

5.1 Utilisation dans les services

Chaque service qui dépend d'un multiplicateur ou d'une feature injecte UniverseContext :

```
// ResearchTimeCalculator.php — applique le multiplicateur de recherche
$time = $baseTime * $this->universeContext->getMultiplier('research_time');

// BuildTimeCalculator.php — applique le multiplicateur de construction
$time = $baseTime * $this->universeContext->getMultiplier('ship_build_time');

// ResourceService.php — applique le multiplicateur de production
$elapsed = $secondsElapsed * $this->universeContext-
>getMultiplier('resource_production');
$stock   = $storedStock + ($productionRate * $elapsed);

// FleetMovementService.php — applique le multiplicateur de déplacement
$travelTime = $distance / ($speed * $speedRatio * $this->universeContext-
>getMultiplier('travel_time'));

// GuildService.php — vérifie si les guildes sont activées
if (!$this->universeContext->isFeatureEnabled('guilds_enabled')) {
    throw new FeatureNotAvailableException('guilds_enabled');
}
```

5.2 Récapitulatif des multiplicateurs

Clé YAML	Impacte	Valeur < 1	Valeur > 1
research_time	ResearchTimeCalculator	Accélère	Ralentit
ship_build_time	BuildTimeCalculator	Accélère	Ralentit
resource_production	ResourceService	Réduit prod.	Augmente prod.
travel_time	FleetMovementService	Accélère	Ralentit

5.3 Récapitulatif des features

Clé YAML	Impacte	Si false
guilds_enabled	GuildService	Création de guilde bloquée
planet_conquest	ColonizationService	Vol de planète impossible

> *La liste des features est extensible. Ajouter une nouvelle feature ne nécessite qu'une clé dans le YAML et un appel isFeatureEnabled() dans le service concerné.*

6. Impact sur la structure du code

6.1 Nouveaux fichiers

```
src/
├── Command/
│   └── UniverseImportCommand.php      # Import JSON → BDD au démarrage
│
├── Entity/
│   ├── Universe.php                   # Définition d'un univers
│   └── PlayerUniverse.php             # Inscription joueur dans un univers
│
├── Manager/
│   └── UniverseManager.php            # Création Universe et PlayerUniverse
│
├── Repository/
│   ├── UniverseRepository.php
│   └── PlayerUniverseRepository.php
│
└── Service/
    └── Universe/
        ├── UniverseContext.php         # Service central — univers courant
        ├── UniverseImporter.php        # Import galaxie/système/planète depuis
JSON
        ├── UniverseGenerator.php       # (existant) génération de la carte
        └── FogOfWarService.php         # (existant)

config/
├── universes/                          # Cartes JSON (généré par Python)
│   ├── omega-1.json
│   └── turbo-1.json
└── game/
    └── universes/                      # Paramètres YAML
        ├── omega-1.yaml
        └── turbo-1.yaml

scripts/
└── universe_generator.py              # Génération des fichiers JSON

docker/
└── entrypoint.sh                      # migrations + import + php-fpm
```

6.2 Modifications sur les entités existantes

Entité	Avant	Après
Player	— (pas de lien univers)	+ current_player_universe: PlayerUniverse|null
Fleet	owner: Player	player_universe: PlayerUniverse
Guild	—	+ universe: Universe
GuildMember	player: Player	player_universe: PlayerUniverse
PlayerTechnology	player: Player	player_universe: PlayerUniverse
ResearchQueue	player: Player	player_universe: PlayerUniverse
AllianceContract	player_a + player_b	player_universe_a + player_universe_b
PlayerDiscovery	player: Player	player_universe: PlayerUniverse
Galaxy	—	+ universe: Universe